

Testra

Complete Technical Reference

An AI system that reads a web codebase and writes runnable end-to-end test suites for Playwright, Cypress, or Selenium — grounded in selectors extracted from the user's real source code.

Repository github.com/Krishna417211/ai-test-generator

Stack Python 3.11+ / FastAPI · React 18 / TypeScript / Vite

Scale ~11,200 lines backend (41 files) · ~7,300 lines frontend (56 files)

Surface 43 HTTP endpoints · 8 SQLite tables · 37 config settings · 353 tests

Generated 15 July 2026

Contents

1. What Testra Is — the product in one page
2. System Architecture — components and data flow
3. The Request Lifecycle — analyze → stream → generate
4. The Two-Agent Pipeline — FilterAgent and WriterAgent
5. The LLM Router — multi-provider rotation and failover
6. The File Extractor — deterministic pre-filtering
7. Authentication and Sessions
8. Email, Verification, and One-Time Secrets
9. The Admin Model
10. Quota and Billing
11. The Security Scanner
12. GitHub Integration and Publishing
13. The Data Model — 8 tables
14. Complete API Reference — 43 endpoints
15. Frontend Architecture
16. Configuration Reference — 37 settings
17. Testing and CI
18. Known Limitations
19. Open Findings — issues identified in review
20. Appendix — full file inventory

1 · What Testra Is

Testra takes a GitHub URL (or an uploaded ZIP) and produces a complete, runnable end-to-end test suite. The product promise is on the landing page: *"Paste a GitHub URL. Get production-ready E2E tests in seconds."*

The problem it solves

Asking a general-purpose LLM to "write E2E tests for my app" produces tests full of invented selectors — `#login-button` where the real element is `[data-testid="submit"]`. The tests look right and fail immediately. Testra's entire architecture is a response to that single failure mode: **ground the model in selectors that provably exist in the source, then verify what it produced against that source and flag anything it invented.**

What it produces

- **Page Object Model classes** — one per key page, not flat scripts
- **Spec files** grouped by page or flow, with positive and negative cases
- **A framework config file** (e.g. `playwright.config.ts`)
- **CI/CD pipelines** — GitHub Actions and GitLab CI YAML
- **Selector warnings** — anything not verifiable against source is marked 

Supported targets

Framework	Languages
Playwright	JavaScript, TypeScript, Python
Cypress	JavaScript, TypeScript
Selenium	Python, Java

Input stacks it recognizes: React, Vue, Next.js, Nuxt, Angular, Svelte on the SPA side; Django, Flask, FastAPI, Rails, Laravel plus their template engines on the server-rendered side.

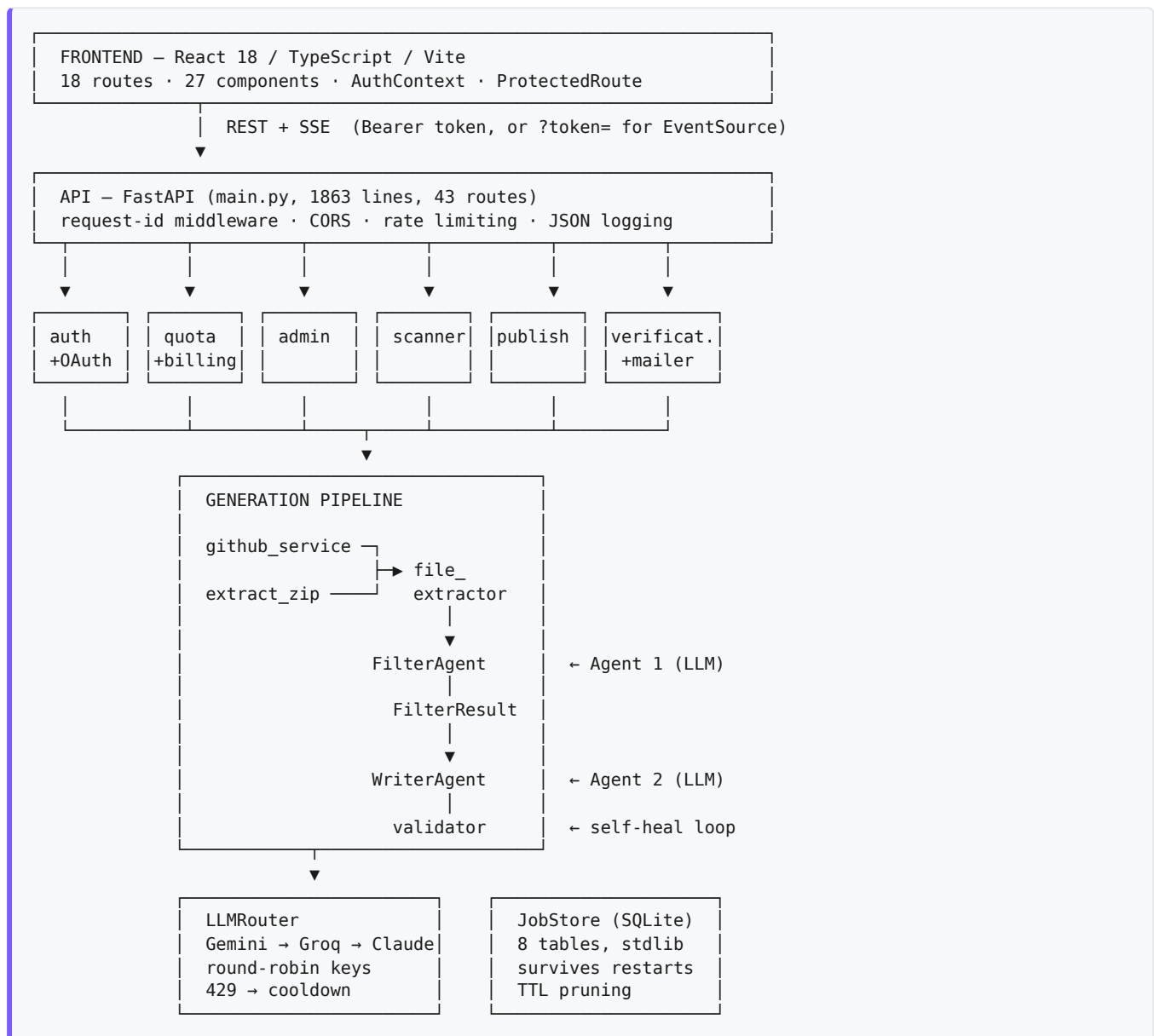
Feature surface beyond test generation

The product has grown past its original single purpose. It now includes:

- **Repo publishing** — push a generated suite into a brand-new GitHub repo in one commit
- **Passive security scanning** — grade a deployed URL's production configuration
- **Accounts** — email/password and GitHub OAuth, email verification, login OTP, password reset
- **Plans and quota** — free tier metered per calendar month, unmetered Pro
- **A user dashboard** — activity feed, per-day charts, published repos
- **An admin console** — user management, plan/quota override, provider health, platform analytics

Design signature. Throughout this codebase, the docstrings record *why* a decision was made and what breaks otherwise — not what the code does. That reasoning is preserved verbatim in this document wherever it explains a non-obvious choice, because in several cases the "obvious" alternative is a security or correctness bug.

2 · System Architecture



Layer responsibilities

Layer	Modules	Owns
API	main.py, models/schemas.py, ratelimit.py	HTTP surface, validation, rate limits, correlation IDs
Identity	auth, github_oauth, verification, mailer, admin	Who the caller is and what they may do
Commerce	quota, billing	Metering, plans, entitlement grants
Pipeline	file_extractor, filter_agent, writer_agent, validator	Turning a repo into tests
Providers	llm_router	Reaching an LLM despite rate limits
Features	security_scanner, git_publisher, github_service	Scanning, publishing, fetching

Persistence	<code>store</code>	Every durable fact, in one SQLite file
Cross-cutting	<code>config</code> , <code>logging_config</code>	Typed settings, structured logs

No external infrastructure. Persistence is `stdlib sqlite3` ; rate limiting is in-process; there is no Redis, no queue, no ORM. The system runs from a single process with no services to provision. The cost is documented in §18: rate limits are per-worker, so multi-worker deployment weakens them.

3 · The Request Lifecycle

Test generation is a three-call flow, deliberately split so the expensive LLM work is visible while it happens.

Step 1 — Analyze `POST /api/analyze`

1. `parse_github_url` splits the URL into owner / repo / branch / subpath
2. `GitHubService.get_file_tree` pulls the whole tree in one Git Trees API call (`recursive=1`)
3. `fetch_files` pulls contents concurrently — `raw.githubusercontent.com` for public repos (no rate limit), the Contents API for private
4. `FileExtractor.extract` filters, scores, and budgets the files — no LLM, fast and free
5. `FilterAgent` asks the LLM for a structured project analysis
6. The job is persisted in SQLite; `job_id` + analysis returns to the client

The alternative entry point, `POST /api/upload-zip` , replaces steps 1–3 with `extract_zip` (in-memory, handles GitHub's nested ZIP layout) and is otherwise identical.

Step 2 — Stream `GET /api/stream/{job_id}` (optional)

Server-Sent Events stream Agent 2's output token by token. The raw output is cached onto the job row, so a subsequent generate call reuses it rather than paying for a second LLM call — the `pregenerated_raw` parameter on `WriterAgent.run` .

Why SSE takes a query-param token. The browser `EventSource` API cannot set custom headers, so it cannot send `Authorization: Bearer . auth._extract_token` therefore accepts `?token=` as well. This is a deliberate, documented concession to a browser limitation, not an oversight.

Step 3 — Generate `POST /api/generate/{job_id}`

1. `require_quota` authenticates and *reserves* a generation credit
2. Ownership is re-checked — a job belonging to another account is 403, a missing one 404
3. `WriterAgent.run` plans the file list, then generates each file
4. `validator.validate_files` syntax-checks the output; failures drive the self-heal loop
5. Selectors are validated against source; unverifiable ones are flagged
6. On success the lease commits; **on failure it refunds** — a crashed run costs no credit

Failed runs are recorded, not dropped. From `main.py` : *"from the dashboard's side a run that vanished and a run that never happened look identical, and the user who watched this spin for a minute deserves to see it happened."* Failures are written to generations with `status='failed'` and surface in the admin console's `recent_failures` view.

4 · The Two-Agent Pipeline

Agent 1 — FilterAgent (agents/filter_agent.py, 238 lines)

Runs in two phases.

Phase 1a — local extraction (no LLM)

`FileExtractor.extract` does the heavy lifting deterministically. See §6.

Phase 1b — LLM project analysis

Sends a compact file-tree summary and asks, in JSON mode, for a `FilterResult`: project summary, key pages with test priorities, key components, routes, and testing challenges.

The `FilterResult` contract — the pipeline's central data structure:

Field	Type	Meaning
<code>project_summary</code>	<code>str</code>	Human-readable summary shown in the UI
<code>key_pages</code>	<code>list[dict]</code>	<code>{path, description, test_priority, suggested_tests}</code>
<code>key_components</code>	<code>list[dict]</code>	<code>{path, description}</code>
<code>routes</code>	<code>list[str]</code>	Routes detected in routing code
<code>framework</code>	<code>str</code>	Detected stack
<code>testing_challenges</code>	<code>list[str]</code>	Obstacles, plus extractor warnings
<code>files</code>	<code>dict[str,str]</code>	The filtered source — passed whole to Agent 2
<code>total_tokens</code> , <code>file_count</code>	<code>int</code>	Extraction accounting

Two anti-hallucination defences, both learned the hard way.

1. NoTestableUIError . If extraction yields zero browser-renderable files, the pipeline stops rather than calling the LLM. The docstring records why: *"with an empty file tree the LLM has nothing to describe, and it answers by echoing the shape of the example in the prompt — inventing a login page, a /dashboard route and a JWT flow for a project that has none. Failing loudly beats a confident fiction."*

2. Placeholder-only schemas. The prompt's JSON schema uses `<angle-bracket>` placeholders, never realistic samples. Again from the source: *"An earlier version illustrated the shape with 'src/pages/Login.tsx', routes ['/', '/login', '/dashboard'] and a JWT/infinite-scroll challenge — and the model reproduced those verbatim for projects that contained none of them. Describe the shape; never supply content that is plausible enough to copy."*

The prompt also carries explicit grounding rules that override everything else: every path must appear verbatim in the file tree, every route must be traceable to shown code, and empty arrays are correct where padded filler is a failure.

Agent 2 — WriterAgent (agents/writer_agent.py, 861 lines)

File-by-file generation

The original design asked for the whole suite as one JSON blob and hit the output-token cap on large repos — silently truncating. The current design is two-stage:

1. `_plan_files` — asks only for a manifest (filenames + roles). A tiny response that cannot truncate. Capped at 12 files.
2. `_generate_one_file` — generates each file's contents in its own call, so every response stays well under the cap.

`_generate_single_shot` survives as a fallback when planning yields nothing.

Selector grounding

`_extract_available_selectors` mines the real source for `data-testid`, `data-cy`, `id`, `name`, and `class` values, and injects them into the prompt under the heading *"prefer these; anything else is a guess."* The model is instructed to mark any selector it must invent with  WARNING: Selector may need verification.

The self-heal loop

Generated files go to `services/validator.py`, which performs a real syntax check where a toolchain exists — `py_compile` for Python, `node --check` for JavaScript — and a structural check (bracket balance, non-truncation) otherwise. Failures feed back for regeneration.

Failure honesty

When a single file fails to generate, the exception is *not* swallowed. From the source: *"Swallowing this used to turn 'the LLM died mid-suite' into a silent gap: the file vanished, the run still looked successful, and CI got pushed against a suite that couldn't run."* Failed filenames are recorded on `_failed_files` and surfaced.

5 · The LLM Router

services/llm_router.py — 703 lines

The router exists so that free-tier rate limits never surface to the user. Its strategy:

- 1. Try providers in priority order
- 2. Within a provider, rotate round-robin through every configured key
- 3. On 429 or error, back off exponentially, then move to the next key or provider
- 4. Track exhausted keys per reset window and auto-restore after cooldown
- 5. Stream live provider status to the frontend over SSE

Provider table

Order	Provider	Model	Context	Max output	Timeout	Key env prefix
1	Gemini	Gemini 1.5 Flash	~1M	8192	60s	GEMINI_API_KEY_1..10
2	Groq	llama-3.3-70b-versatile	128k	4096	30s	GROQ_API_KEY_1..10
3	Claude	claude-haiku-4-5	200k	4096	120s	ANTHROPIC_API_KEY_1..10

Gemini is first because of its 1M context — the source comment reads `# 1M context – best for large repos`.

Key concepts

Type	Role
APIKey	One credential. <code>is_available</code> , <code>mark_exhausted</code> , <code>record_success</code> .
ProviderState	All keys for one provider. <code>next_available_key</code> round-robins, skipping exhausted.
LLMRouter	The engine. Thread-safe via asyncio locks — safe under concurrent FastAPI requests.
<code>is_hard_quota</code>	Distinguishes "out of credit / daily quota burned" from a transient 429.

Exceptions

- `RateLimitError` — 429 or signalled quota exhaustion
- `ProviderError` — non-rate-limit failure (5xx, malformed response)
- `AllProvidersExhausted` — every provider and key tried and failed

Per-request status isolation. Status callbacks are scoped with a `ContextVar` keyed to the current async task, so concurrent SSE streams never cross-talk. `subscribe_status` registers a callback for the current task only.

Exhaustion is an outage, not an upsell. When every key is hard-blocked, capacity is gone for everyone — Pro subscribers included. `quota.py` records the reasoning: *"Upselling at that moment would be selling something we cannot deliver."* That path stays a plain 503; only genuine per-user limits return 402.

Observability

`get_status` feeds the public status panel. `get_key_health` feeds the admin console with per-key state, including which env var to rotate when one goes bad. `get_call_log` exposes recent calls; `_log_call` records provider, key, context hint, latency, and token usage per call.

6 · The File Extractor

services/file_extractor.py — 639 lines

The extractor is where most of the intelligence lives, and it costs nothing — no LLM call. Its stated core insight: *"we only want files that render in the browser. Backend logic, DB migrations, server configs — all noise."*

The scoring system

Score	Category
10	Router files (<code>App.jsx</code> , <code>router/index.js</code>) — always included
9	Page components (<code>pages/</code> , <code>views/</code> , <code>screens/</code>)
8	Top-level layout components
7	Form and auth components
6	Shared UI components
5	Config files (<code>package.json</code> , <code>vite.config.js</code>)
4	Utility components
1	Everything else UI-adjacent

Budget behaviour

Files are admitted in score order until `token_budget` (default **600,000**) is reached. Crucially, low-value files are **summarized rather than dropped** — `_summarize_file` keeps only the testing-relevant parts: element IDs, class names, form fields, button text, route definitions. Token estimation is `len(content) // 4`, a rough 4-chars-per-token heuristic.

Other responsibilities

- `detect_framework` — SPA frameworks, server-rendered backends, and template engines
- `detect_monorepo` — identifies sub-packages
- `find_manifest` — shallowest file of a given name, at any depth
- `extract_zip` — in-memory ZIP handling, including GitHub's nested layout
- `filter_for_push` — what belongs in a fresh repo, with a 1 MB per-file cap (`PUSH_MAX_FILE_BYTES`)

7 · Authentication and Sessions

services/auth.py — 208 lines

Sessions are **opaque server-side tokens**, stored in SQLite, not JWTs. The reason is stated plainly: so they are revocable. A JWT cannot be un-issued; a row can be deleted.

Token acceptance

`_extract_token` accepts `Authorization: Bearer` or a `?token=` query parameter. The latter exists solely because `EventSource` cannot set headers (§3).

Function surface

Function	Role
<code>hash_password / verify_password</code>	Password hashing
<code>validate_email / validate_password</code>	Input rules
<code>encrypt_secret / decrypt_secret</code>	Fernet encryption for stored GitHub tokens. No-op if <code>SESSION_SECRET</code> is unset; legacy plaintext (no prefix) reads back as-is.
<code>create_login_session</code>	Mints an opaque token
<code>public_user</code>	Strips secrets before a user is sent to the client
<code>is_admin</code>	Whether the account holds the admin role
<code>require_user</code>	The FastAPI dependency — token → user, or 401

`require_user` returns a context dict: `{user, session, session_id, user_id}`.

Session lifetime

`touch_session` slides expiry forward on use — *an idle timeout, not a fixed one*.

`delete_sessions_for_user` revokes every session for an account, optionally keeping the current one (the "log out everywhere else" behaviour).

GitHub OAuth (services/github_oauth.py, 93 lines)

- `authorize_url()` — send the browser to GitHub's consent screen
- `exchange_code()` — swap `?code` for an access token
- `get_user()` — resolve login, name, avatar

Lets a user authorize repo creation without pasting a Personal Access Token. Requires `GITHUB_CLIENT_ID` / `GITHUB_CLIENT_SECRET`.

Rate limiting (ratelimit.py, 57 lines)

In-memory per-IP sliding windows, applied as FastAPI dependencies. Used via

`Depends(rate_limit(...))` rather than a decorator — the module notes this is deliberate, because *unlike slowapi's decorator, it works correctly on routes with path params*.

Limiter	Applies to
<code>analyze_limiter</code>	Repo analysis, ZIP upload, repo delete
<code>generate_limiter</code>	Generation and streaming

email_limiter	6 per hour per IP — anything that sends mail
otp_limiter	OTP submission attempts

The email limiter is deliberately much tighter than the rest: *"each call puts a message in someone else's inbox, so an unlimited one is a free mail-bombing service pointed at any address an attacker names, and it burns the relay's reputation and sending quota along the way."*

8 · Email, Verification, and One-Time Secrets

The mailer (services/mailer.py, 112 lines)

Transport only, over `aiosmtpLib`. Async so a send never blocks the event loop the way `stdlib smtpLib` would. `_assert_header_safe` rejects CR/LF in anything that becomes a header — header-injection defence.

Two exceptions distinguish the failure modes: `EmailNotConfigured` (no relay set up) and `EmailDeliveryError` (relay present but refused or dropped).

Dev without SMTP. With no relay configured outside production, the mailer *logs* the message instead of sending. The full signup → verify → OTP flow runs locally with no mail account. In production an empty relay is **refused rather than skipped**, so the checks cannot silently disable themselves.

Verification (services/verification.py, 253 lines)

This module carries the most carefully reasoned threat model in the codebase.

Why the two kinds of secret are stored differently.

Link tokens (verify, reset) are 256-bit random. *"Guessing one is not a thing that happens, so a plain SHA-256 is enough to keep the database from being a list of working links: it's preimage-resistant, and there's no dictionary to run against a value with that much entropy."*

The login OTP is six digits — a million possibilities, which a GPU chews through instantly. It therefore gets **bcrypt**, not SHA-256.

The asymmetry is the point: hash strength is chosen against the entropy of what is being hashed, not applied uniformly.

Function	Role
<code>send_verification_email</code> / <code>consume_verification_token</code>	Signup confirmation
<code>send_password_reset_email</code> / <code>consume_reset_token</code>	Password reset
<code>start_login_challenge</code> / <code>verify_login_challenge</code>	Login OTP, issued only after the password check passes
<code>_new_code</code>	6-digit code from a CSPRNG
<code>mask_email</code>	<code>ada@example.com</code> → <code>a**@example.com</code>

Two more deliberate choices:

- `InvalidCode` is one exception for *wrong, used, and expired*. *"Telling an attacker which is a free oracle."*
- `_new_code` uses `secrets`, never `random` — *"the latter's Mersenne Twister is reconstructible from past outputs."*

TTLs and limits

Setting	Default
---------	---------

email_verify_ttl_seconds	86,400 (24h)
password_reset_ttl_seconds	3,600 (1h)
login_otp_ttl_seconds	600 (10m)
login_otp_max_attempts	5 per challenge

9 · The Admin Model

services/admin.py — 57 lines. The smallest security-critical module, and the most opinionated.

How someone becomes an admin

By being listed in `ADMIN_EMAILS` with a **verified** address. That is the entire mechanism. There is deliberately no role column, no grant table, and no promote-user endpoint.

The reasoning, verbatim from the source:

- privilege can't be escalated by writing to the database, because the grant isn't in the database;
- there is no privileged-write endpoint to find a bug in, because there is no endpoint;
- a stolen admin session can act as that admin until it expires, but cannot mint a second admin.

The trade is explicit: changing the admin list is a config change plus a restart — *"the right friction for the one role that can read every account."*

Two rules the module exists to enforce

1. **The address must be verified.** With `REQUIRE_EMAIL_VERIFICATION=false` a stranger could sign up as an admin's address and be issued a session — so the allowlist alone is not proof of who holds the address.
2. **Non-admins get 404, not 403.** *"A 403 confirms the console exists to someone who already has a session; to everyone else it should look absent."*

The client's `user.is_admin` decides whether the nav item renders and **nothing else** — every route re-derives the role server-side via `require_admin`.

Admin capabilities

Capability	Endpoint
List / search users	GET <code>/api/admin/users</code>
User detail	GET <code>/api/admin/users/{id}</code>
Suspend / unsuspend	POST <code>/api/admin/users/{id}/suspend</code>
Override plan	POST <code>/api/admin/users/{id}/plan</code>
Override usage counter	POST <code>/api/admin/users/{id}/usage</code>
Revoke all sessions	POST <code>/api/admin/users/{id}/logout-all</code>
Delete account	DELETE <code>/api/admin/users/{id}</code>
Platform overview	GET <code>/api/admin/overview</code>
Provider / key health	GET <code>/api/admin/system</code>

`set_usage` exists as a support tool — *"your run failed, here's the credit back."* `set_suspended` notes that sessions are the caller's job: a suspended user holding a live token would otherwise keep working.

10 · Quota and Billing

Quota (services/quota.py, 166 lines)

Free accounts get `free_generations_per_month` (default **5**) generations per **UTC calendar month**. Pro is unmetered.

The lease pattern

Quota is **reserved before** the work starts and **refunded if it fails**, so a crashed run doesn't cost the user a credit.

Piece	Role
<code>current_period</code>	The billing bucket — UTC calendar month
<code>next_period_start</code>	Epoch seconds at which usage resets
<code>limit_for</code>	Allowance per month, or <code>None</code> for unlimited
<code>get_quota</code>	Read-only — consumes nothing
<code>QuotaLease</code>	A reserved unit. <code>.commit()</code> or <code>.refund()</code> .
<code>consume_quota</code>	Reserve one, or raise 402
<code>require_quota</code>	The dependency: authenticate, then reserve

At the store layer, `try_consume` is **atomic** — the reservation cannot double-spend under concurrency. `get_plan` resolves the *effective* plan: a lapsed paid plan reads as `free` without needing a background job to downgrade it.

Billing (services/billing.py, 166 lines)

This module owns *what a plan costs* and *how a plan gets granted*. It deliberately does **not** implement a payment provider.

The entitlement rule. An entitlement may only be granted by something that **proves a payment happened for a known user** — a hosted checkout plus a signed webhook. `set_plan`'s docstring is explicit: *"Call this from a verified payment webhook — never straight from a client request."*

Function	Role
<code>plans</code>	The catalogue the pricing UI renders
<code>checkout_url</code>	Hosted link, tagged so the webhook can identify the payer
<code>verify_webhook</code>	Constant-time signature check over the raw body
<code>is_granting_event</code>	False for <code>payment.failed</code> , <code>subscription.cancelled</code> , refunds, and the many other webhook types that must not unlock anything
<code>extract_grant</code>	Pulls <code>(user_id, plan_id)</code> from a verified payload

Wiring is provider-agnostic: put the links in `BILLING_CHECKOUT_URL_MONTHLY` / `_YEARLY` and the signing secret in `BILLING_WEBHOOK_SECRET`. Razorpay and Stripe both work, and both support UPI for Indian customers. Defaults are \$20/month, \$100/year.

11 · The Security Scanner

services/security_scanner.py — 628 lines

Given a URL the user owns, the scanner fetches the site and inspects its **configuration**. It is explicitly non-intrusive: it reads headers, cookies, TLS/redirect behaviour, and probes a short curated list of files that should never be public. **It sends no attack payloads and brute-forces nothing.**

Checks performed

Check	Covers
<code>_check_transport</code>	HTTPS enforcement, HSTS
<code>_check_security_headers</code>	CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy
<code>_check_cookies</code>	Secure, HttpOnly, SameSite — reported per <i>problem</i> , not per cookie
<code>_check_disclosure</code>	Server / framework version leakage
<code>_check_cors</code>	CORS misconfiguration
<code>_check_content</code>	Content-level issues
<code>_check_exposed_files</code>	<code>.env</code> , <code>.git/config</code> , and similar

SSRF defence, in two layers.

`_assert_public_host` resolves the hostname and rejects private, loopback, link-local, reserved, and multicast IPs.

`_redirect_guard` re-validates **every redirect hop** as an httpx response hook, firing *before* each redirect is followed — so an open redirect on the target cannot bounce the scanner onto an internal address. Validating only the initial URL would leave exactly that hole.

Scoring

Findings roll into a 0-100 score and a letter grade. Penalties are **capped per category**, because a plain sum let one repeated misconfiguration sink everything: *"five cookies each missing three flags is -120 on a 100-point scale, which graded any site with a few analytics cookies F no matter how good the rest of it was."*

`exposure` is uncapped on purpose: *"a world-readable .env is not one issue among several, and it should be able to take the grade to F by itself."*

`_looks_sensitive` confirms a 200 response actually resembles the sensitive file rather than an SPA catch-all returning 200 for everything — the difference between a real finding and a false positive on every single-page app.

`_youtube` generates a YouTube *search* URL for each fix, never a specific video — a link that can't rot into someone else's content.

12 · GitHub Integration and Publishing

Fetching (services/github_service.py, 230 lines)

Path	Mechanism
Public repos	<code>raw.githubusercontent.com</code> — no auth, no rate limits, faster
Private repos	GitHub REST Contents API with OAuth token or PAT
ZIP upload	<code>file_extractor.extract_zip</code>

`get_file_tree` uses the Git Trees API with `recursive=1` — the whole tree in one request.
`parse_github_url` handles bare repo URLs, `/tree/branch`, and `/tree/branch/subpath`, and its test suite pins tricky cases: `krishna.github.io`, `awesome.gitignore-demo`, `my.github.io.git` — i.e. stripping `.git` without mangling names that legitimately contain it.

Errors: `RepoNotFoundError`, `RepoAccessError`.

Publishing (services/git_publisher.py, 265 lines)

Creates a fresh GitHub repo and pushes a whole project into it in a **single commit**, using the Git Data API end-to-end — **no local git binary required**.

1. GET `/user` → resolve authenticated owner
2. POST `/user/repos (auto_init)` → create repo with a base commit
3. GET `/git/ref/heads/{branch}` → base commit sha
4. GET `/git/commits/{sha}` → base tree sha
5. POST `/git/blobs (per file)` → blob shas, uploaded concurrently
6. POST `/git/trees` → new tree layered on the base
7. POST `/git/commits` → new commit
8. PATCH `/git/refs/heads/{branch}` → move the ref

Publish-then-generate ordering. `main.py` notes that tests are generated and validated locally — through `WriterAgent`'s self-heal loop — *before* the push. The repo is only created once there is something worth pushing.

`was_published_by` is the authorization gate on deletion: **true only if THIS user published THIS repo through the app**. Testra will not delete a repo it did not create on that user's behalf, even with a valid token that could. `delete_repo` is marked irreversible.

13 · The Data Model

services/store.py — 1,052 lines. Stdlib `sqlite3` only; thread-safe; no ORM.

Tables

Table	Columns
<code>users</code>	id, email, password_hash, name, github_id, github_login, avatar_url, created_at, plan, plan_expires_at, email_verified, suspended, suspended_at, suspended_reason
<code>sessions</code>	session_id, created_at, expires_at, data
<code>jobs</code>	job_id, created_at, updated_at, data
<code>usage</code>	user_id, period, count
<code>generations</code>	id, user_id, created_at, source, framework, language, test_count, file_count, status, duration_ms, error
<code>scans</code>	id, user_id, created_at, url, grade, score, findings, duration_ms, counts
<code>published_repos</code>	full_name, user_id, repo_url, created_at, test_count, files_pushed, cicd_added, private
<code>user_settings</code>	user_id, framework, language, base_url, updated_at

Lifecycle management

- `_migrate` — adds columns to tables that already exist in deployed databases
- `prune` / `prune_sessions` — TTL expiry (`job_ttl_seconds` , default 24h)
- `vacuum` — reclaims file space, *"SQLite keeps pages otherwise"*
- `sweep` — expire stale jobs and sessions, then reclaim

Notable method semantics

Method	Why it's written the way it is
<code>activity_totals</code>	Lifetime counts, deliberately separate from the capped <code>list_*</code> methods — <i>"the headline number must not silently"</i> track a display cap
<code>activity_feed</code>	Every feature's activity as one timeline, newest first
<code>activity_series</code>	Per-day counts, zero-filled
<code>platform_series</code>	The all-tenant twin of <code>activity_series</code> , zero-filled the same way
<code>_escape_like</code>	Neutralises LIKE wildcards so a search term matches itself literally
<code>try_consume</code>	Atomic quota reservation
<code>get_plan</code>	Effective plan — lapsed paid reads as free with no background job
<code>delete_account</code>	Erases a user and everything belonging to them. Irreversible.
<code>recent_failures</code>	Failed generations across all users — <i>"the admin's first stop when something is broken"</i>

14 · Complete API Reference

43 routes. All under `main.py`.

System

Method	Path	Purpose
GET	<code>/health</code>	Liveness
GET	<code>/metrics</code>	Metrics
GET	<code>/api/status</code>	Provider health for the public status panel
GET	<code>/api/logs</code>	Recent router call log

Generation pipeline

Method	Path	Guards
POST	<code>/api/analyze</code>	<code>analyze_limiter</code>
POST	<code>/api/upload-zip</code>	<code>analyze_limiter</code>
POST	<code>/api/generate/{job_id}</code>	<code>require_quota + ownership</code>
GET	<code>/api/stream/{job_id}</code>	<code>generate_limiter · SSE</code>

Authentication

Method	Path	Purpose
GET	<code>/api/auth/config</code>	Which auth methods this server offers
POST	<code>/api/auth/signup</code>	Register
POST	<code>/api/auth/login</code>	Password check → session or OTP challenge
POST	<code>/api/auth/login/verify-otp</code>	Complete OTP login
POST	<code>/api/auth/verify-email</code>	Redeem a verification link
POST	<code>/api/auth/resend-verification</code>	<code>email_limiter</code>
POST	<code>/api/auth/forgot-password</code>	<code>email_limiter</code>
POST	<code>/api/auth/reset-password</code>	Redeem a reset token
POST	<code>/api/auth/change-password</code>	Authenticated change
POST	<code>/api/auth/logout</code>	End this session
POST	<code>/api/auth/logout-all</code>	Revoke every session
GET	<code>/api/auth/github/login</code>	OAuth start
GET	<code>/api/auth/github/callback</code>	OAuth return
GET	<code>/api/auth/me</code>	Current user

Account

Method	Path	Purpose
GET	<code>/api/profile</code>	Profile + lifetime activity

GET	/api/dashboard	Feed, series, published repos
GET / PUT	/api/settings	Form defaults
DELETE	/api/account	Irreversible erase

Admin

Method	Path
GET	/api/admin/users · /api/admin/users/{id}
POST	/api/admin/users/{id}/suspend · /plan · /usage · /logout-all
DELETE	/api/admin/users/{id}
GET	/api/admin/overview · /api/admin/system

Billing

Method	Path	Note
GET	/api/billing/me	Plan + quota state
POST	/api/billing/checkout	Hosted checkout link
POST	/api/billing/webhook	The only path that grants entitlement. Signature-verified.

Publishing and scanning

Method	Path	Note
POST	/api/publish-zip	Create repo + push
GET	/api/repos	Repos published through Testra
DELETE	/api/repo/{owner}/{repo}	Gated by was_published_by
POST	/api/scan	Passive security scan

15 · Frontend Architecture

React 18 · TypeScript · Vite · React Router · 56 files, ~7,300 lines

Routes

Path	Page	Access
/	Home	Public
/login · /signup	Login / Signup	Public
/auth/callback	AuthCallback	Public
/verify-email	VerifyEmail	Public
/forgot-password · /reset-password	ForgotPassword / ResetPassword	Public
/status	Status	Public
/dashboard	Dashboard	Protected
/generate	Generate	Protected
/publish	Publish	Protected
/scan	Scan	Protected
/settings · /profile	Settings / Profile	Protected
/admin · /admin/users · /admin/system	Admin console	Admin
*	NotFound	Public

State and access control

- context/AuthContext.tsx — the single source of session state
- components/ProtectedRoute.tsx — redirects unauthenticated users
- components/ErrorBoundary.tsx — catches render failures
- nav.ts — nav model, including the admin item gated on user.is_admin

Client-side gating is cosmetic by design. user.is_admin decides only whether the nav item renders. Every admin route re-derives the role server-side. The frontend check is UX; the server check is security.

Component groups

Group	Components
Generation flow	GenerateInput, ConfigureStep, PreviewStep, ResultsStep, StepIndicator, StreamingOutput, CodeViewer
Dashboard	DashboardLayout, ActivityChart, CountUp
Admin	AdminLayout, AdminLogin, AdminOverview, AdminUsers (547 lines), AdminSystem
Auth	AuthForm, ProtectedRoute
Features	PublishPanel, ScanPanel, ProviderStatus, UpgradeModal
Presentation	Hero3D, AuroraBackground, TiltCard, ScrollProgress, Page, PageHeader, Footer, NavBar

Utilities

- `utils/api.ts` (731 lines) — the entire typed API client
- `types/index.ts` (237 lines) — shared types mirroring the backend schemas
- `utils/format.ts` , `utils/download.ts` , `utils/chartPalette.ts` , `lib/celebrate.ts`

Build output

`npm run build` runs `tsc && vite build`. Current output is ~631 KB main bundle (187 KB gzipped) plus a separate ~828 KB `Hero3D` chunk (224 KB gzipped). Vite warns that both exceed the 500 KB threshold — the 3D hero is the dominant cost and is a code-splitting candidate.

16 · Configuration Reference

config.py — typed via `pydantic-settings`. Read from the environment and `backend/.env` in one place, rather than scattered `os.getenv` calls.

Application

Setting	Default
<code>app_env</code>	<code>development</code>
<code>log_level</code>	<code>INFO</code>
<code>cors_origins</code>	<code>*</code>
<code>frontend_url</code>	<code>http://localhost:5173</code>
<code>backend_url</code>	<code>http://localhost:8000</code>

Sessions and OAuth

Setting	Default
<code>github_client_id</code> / <code>github_client_secret</code>	empty
<code>oauth_session_ttl_seconds</code>	28,800 (8h)
<code>session_secret</code>	empty — tokens stored unencrypted if unset (local dev only)

Email

Setting	Default
<code>smtp_host</code> / <code>smtp_user</code> / <code>smtp_password</code> / <code>smtp_from</code>	empty
<code>smtp_port</code>	587
<code>smtp_from_name</code>	<code>Testra</code>
<code>smtp_starttls</code> / <code>smtp_ssl</code>	true / false
<code>smtp_timeout</code>	15
<code>require_email_verification</code>	true
<code>login_otp_enabled</code>	true

Limits and pipeline

Setting	Default
<code>max_repo_size_mb</code>	100
<code>max_files_per_repo</code>	2,000
<code>token_budget</code>	600,000 — see §19
<code>job_ttl_seconds</code>	86,400 (24h)

Plans

Setting	Default
---------	---------

<code>free_generations_per_month</code>	5
<code>pro_price_monthly_usd</code> / <code>_yearly_usd</code>	20 / 100
<code>billing_checkout_url_monthly</code> / <code>_yearly</code>	empty
<code>billing_webhook_secret</code>	empty
<code>admin_emails</code>	empty — the entire admin grant

Derived properties

`is_production`, `cors_origin_list`, `admin_email_list`, `is_admin_email`, `admin_enabled`, `github_oauth_enabled`, `smtp_configured`, `mail_from_address`, `email_enabled`, `billing_enabled`, `oauth_callback_url`. Admin emails are compared lowercased, because that is how `users.email` is stored.

17 · Testing and CI

Suite

353 tests across 15 files, all passing.

File	Lines	Covers
test_admin.py	442	Admin grant, verification requirement, 404-not-403
test_security_scanner.py	420	Checks, scoring caps, SSRF guards
test_file_extractor.py	405	Filtering, scoring, framework detection
test_verification.py	339	Tokens, OTP, expiry, attempt caps
test_store.py	253	Persistence, migrations, pruning
test_quota.py	192	Leases, refunds, period rollover
test_billing.py	148	Webhook verification, grant extraction
test_git_publisher.py	137	Blob→tree→commit→ref flow
test_llm_router.py	135	Failover, rotation, cooldown
test_filter_agent.py	132	Agent 1 behaviour
test_writer_agent.py	90	Selector extraction/validation, JSON parsing
test_auth.py	90	Hashing, validation, token extraction
test_github_service.py	69	URL parsing edge cases
test_validator.py	47	Syntax checks
test_ratelimit.py	24	Sliding window

The new admin and verification tests import `services.admin` / `services.verification` directly rather than through `main.py`, so they exercise the modules without standing up the app.

CI (`.github/workflows/ci.yml`)

Job	Runs
Backend	Matrix Python 3.11 / 3.12 / 3.13 · <code>ruff check .</code> · <code>pytest --cov</code>
Frontend	Node 20 · <code>npm run build</code> (tsc + vite)

Triggers: push to `master` / `main` / `develop`; PRs to `master` / `main`.

18 · Known Limitations

From `docs/LIMITATIONS.md` — the project's own model card. Reproduced because it is load-bearing for anyone trusting the output.

What "validated" means — and doesn't. The green *"passed a syntax check"* badge means each generated file **parses**. It does **not** mean the tests were executed, that selectors resolve against a running app, or that assertions pass. Always run the suite against your app before trusting it in CI.

Limitation	Detail
Selector hallucination	Extraction is regex-based today. It can miss dynamic classes (<code>className={styles.x}</code>), CSS-in-JS, and ARIA/role/text selectors — so both false positives and false negatives are possible.
Large repos	May exceed the model's context window. Low-priority files are summarized rather than sent verbatim, which reduces selector accuracy.
Output size	Extremely large suites can hit provider output-token limits. Narrow the scope and regenerate.
Free-tier variance	Quality and latency depend on whichever provider is available. Results can vary run to run.

Data handling

- Repo contents are held only for a job's lifetime (SQLite, pruned after 24h) and sent to the selected LLM provider
- Linked GitHub tokens are encrypted at rest **when** `SESSION_SECRET` is set
- The scanner is passive and refuses internal targets, including on redirect hops

Operational limits not in the model card

- **Rate limiting is per-worker.** In-memory windows mean N workers permit roughly N× the intended rate. `ratelimit.py` flags Redis as the production fix.
- **Admin changes need a restart** — an accepted trade, not a bug (\$9).

Stated roadmap

- Execute generated suites in a sandbox and report real pass/fail
- AST-based selector extraction (roles, accessible names, visible text)
- Retrieval + per-provider context budgeting for large repos

19 · Open Findings UNRESOLVED

Identified during review of the generation pipeline. These are not in the project's own docs. Estimates are from code reading, not measurement.

19.1 — The token budget exceeds two of three providers' context windows

Severity: high. This is a correctness issue, not an optimization.

`token_budget` defaults to **600,000** (`config.py` , `file_extractor.py:336`). The router's fallback chain is:

Provider	Context	Fits 600k?
Gemini 1.5 Flash	~1M	Yes
Groq llama-3.3-70b	128k	No
Claude Haiku 4.5	200k	No

`MAX_OUTPUT_TOKENS` (`llm_router.py:160`) caps output per provider. **There is no equivalent input guard anywhere.** The budget was sized for Gemini — the source comment at `llm_router.py:147` says so explicitly (`# 1M context – best for large repos`) — but nothing prevents that same oversized prompt reaching a fallback that cannot hold it.

Consequence: on a repo near the budget, Gemini succeeds and both fallbacks fail on context length. The rotation that exists specifically to survive rate limits cannot fall back. Worse, `main.py` catches the resulting `AllProvidersExhausted` and reports it as *"our shared API keys are dry"* — an outage message for what is actually a prompt-length problem.

Compounding: the estimator is `len(content) // 4` , which undercounts code — so true token counts run *above* the budget's own accounting.

Suggested fix: a per-provider input budget alongside `MAX_OUTPUT_TOKENS` , and a distinct error for "prompt too large for every remaining provider" so it stops being misreported as an outage.

19.2 — The full repo source is re-sent once per generated file

Severity: high (cost/latency).

`writer_agent.py:610` embeds `json.dumps(filter_result.files, indent=2)` — the entire extracted source — into every single-file prompt. That prompt is built inside the `for spec in plan:` loop (`writer_agent.py:497`), and `_plan_files` returns up to 12 files. The full source is therefore sent **up to 12 times per generation**, byte-identical each time.

Suggested fix: have `_plan_files` return the source paths each planned file actually depends on, and send only those. A page object for `LoginPage` does not need every component in the repo. Estimated 5–8× reduction in input tokens, on every provider — and it brings prompts back inside the fallback context windows in 19.1.

19.3 — Prompt ordering makes caching impossible

Severity: medium (cost only).

Prompt caching is a prefix match. In `_generate_one_file` , the only content that varies across the 12 calls is `spec['filename']` and its description — and it sits at the **top** of the prompt (`## FILE TO WRITE` , `writer_agent.py:588`). Everything after it is identical every time. Because the one volatile field is at position zero, there is no reusable prefix at all.

Suggested fix: move that block to the end, then add a `cache_control: {type: "ephemeral"}` breakpoint. Claude Haiku 4.5's minimum cacheable prefix is 4,096 tokens — the source block clears that

easily. Cache reads cost $\sim 0.1\times$ base input; writes $1.25\times$ at the default 5-minute TTL.

Caching reduces cost, not tokens. The same tokens are still sent and counted — only the price of the repeated prefix changes. And since caches are per-model and Claude is *third* in rotation, it only pays off on runs Claude actually serves. 19.2 is the provider-agnostic fix; this one compounds with it.

19.4 — Minor

- **Redundant payload.** `_extract_available_selectors` (`writer_agent.py:574`) already distills selectors from the source, yet the prompt sends *both* the distilled list and the full raw source.
- **Wasted whitespace.** `indent=2` on `writer_agent.py:592, 598, 610` adds pure formatting tokens the model gains nothing from.
- **Docstring/behaviour drift.** `filter_agent._build_file_tree_summary` claims to show structure "without sending all file contents", but `any(hp in path for hp in ["pages/", "router"])` (`filter_agent.py:232`) matches *every* file under `pages/` , each contributing an 800-char preview. Bounded per file, unbounded in file count.
- **Bundle size.** The `Hero3D` chunk is ~ 828 KB (224 KB gzipped) and trips Vite's 500 KB warning.

20 · Appendix — File Inventory

Backend — 41 files, ~11,200 lines

File	Lines	Role
main.py	1863	FastAPI app, 43 routes
services/store.py	1052	SQLite persistence, 8 tables
agents/writer_agent.py	861	Agent 2
services/llm_router.py	703	Provider rotation
services/file_extractor.py	639	Deterministic pre-filter
services/security_scanner.py	628	Passive scan
models/schemas.py	349	Pydantic request/response models
services/git_publisher.py	265	Git Data API push
services/verification.py	253	One-time secrets
agents/filter_agent.py	238	Agent 1
services/github_service.py	230	Repo fetching
services/auth.py	208	Sessions, hashing, dependency
config.py	172	Typed settings
services/quota.py	166	Per-user metering
services/billing.py	166	Plans, entitlement
services/mailer.py	112	SMTP transport
services/validator.py	108	Syntax checks
services/github_oauth.py	93	OAuth handshake
ratelimit.py	57	Sliding windows
services/admin.py	57	The admin role
logging_config.py	37	JSON logs + request IDs
tests/*	2923	15 files, 353 tests

Frontend — 56 files, ~7,300 lines

File	Lines
utils/api.ts	731
pages/AdminUsers.tsx	547
pages/Settings.tsx	464
pages/Profile.tsx	388
pages/Dashboard.tsx	293
pages/AdminSystem.tsx	270
components/AuthForm.tsx	260

types/index.ts	237
pages/AdminLogin.tsx · AdminOverview.tsx	234 · 224
components/ResultsStep.tsx	224
components/PublishPanel.tsx	216
components/ActivityChart.tsx	214
components/UpgradeModal.tsx	205
components/ScanPanel.tsx	191
components/ConfigureStep.tsx	182
Remaining 40 files	<170 each

Repository layout

```
testgen-ai/
├── backend/
│   ├── agents/          filter_agent · writer_agent
│   ├── models/         schemas
│   ├── services/       16 modules
│   ├── tests/          15 files, 353 tests
│   ├── main.py · config.py · ratelimit.py · logging_config.py
│   └── requirements.txt · .env.example
├── frontend/
│   └── src/ components/ · pages/ · context/ · utils/ · lib/ · types/
├── docs/
│   └── ARCHITECTURE.md · LIMITATIONS.md
├── .github/workflows/   ci.yml
├── docker-compose.yml
└── README.md
```